EE382V - Systems Programming
Project 2: A yash Daemon

Due Date: **Sunday 10/18 11:59pm**

Objective

In this project you will write client and server programs that mimic the sshd daemon and ssh client. Your daemon will be a yash daemon and the client a yash client.

Features

yashd - The Server

- Your daemon will extend the shell you wrote for project1 so it will be made into a service that clients can access at a well defined port number (**3820**) on a target machine. All the client needs is the IP address and the port at which the service implemented using a TCP/IP socket is running.
- The implementation of the server must mimic a daemon, accordingly it must follow the guidelines for a daemon discussed in class
- The server is run as follows:
  `$ yashd`
- The `yashd` service must be responsive and therefore be implemented as a multi-threaded server using the `pthreads` library
- Each client is accepted and served in a thread. Therefore, the code inside the thread function will be mostly derived from project1.
  - The actual execution of commands will no longer send the output to the stdout but communicated over the socket connection back to the client.
  - Also the commands themselves are still executed using a fork-exec mechanism
- In addition to launching and running commands, the threads must also perform logging. The log entries must be made into a file named `/tmp/yashd.log` in syslog-like format:
  `<date and time><blank>yashd[<Client IP>:<Port>]: <Command executed><newline>`
  Here are two entries:
  `Sep 18 10:02:55 yashd[127.0.0.1:7813]: ls -l`
  `Sep 18 10:04:45 yashd[127.0.0.1:7815]: echo $PATH`
  Note that these are commands that came from two different yash clients (with different port numbers) both running on the localhost.

yash - The client

- The implementation of the client is much simpler in that it mimics the terminal where the user types her commands and gets responses back.
- The client is run as follows:
  `$ yash <IP_Address_of_Server>`
  Note that the port is implicit (**3820**) so the clients can hardcode it.
- The client must have signal handling capabilities to handle the following:
  - `Ctrl-c` - To stop the current running command (on the server)
  - `Ctrl-z` - To suspend the current running command (on the server)
- Terminating the client is still done using `Ctrl-d` or quit

yash - The Protocol

To make it possible to run your client with somebody else's server and vice-versa, we require that your implementation adhere to the following simple protocol:

- The client can send these three types of messages to the server:
  - `CMD<blank><Command_String>\n`
    The start of this type of message is the keyword `CMD` that tells the server to run a command, the command itself follows after a blank space ending with a newline. Here are two examples:
    `CMD ls -l\n`
    `CMD ps -ef | more\n`
  - `CTL<blank><char[c|z|d]>\n`
    The start of this type of message is the keyword `CTL` that tells the server that, what follows after a blank space is one character, which can be `c` or `z` or `d`. This message is also terminated by a newline.
  - Plain text
    This is when the client types in something to the stdin of the command being run. For example if one issues the command:
    ```
    cat > file.txt
    ```
    the text the user types in must be redirected to the file. The typed text is on the client side and is sent as plain text to the server thread handling this shell instance. The end of this plain text must be an EOF character
- The server sends only one type of message to the client:
  - Plain ASCII text which is the resulting output from running a command or control message sent by the client. Note that this text may have multiple newline characters in it.

Implementation Notes

- The command prompt (#) that the client displays should come from the server. The client simply reads and displays it. This solves the need for a special message from the server to the client indicating the end of text from running a command. Therefore,

make sure that the prompt message the server sends is formatted as "\n#". This will ensure that no matter how the output of the last command ends, the prompt always appears on a new line by itself.

- Use the reusePort function given in the server example to make sure you are able to bind to the same port each time the daemon runs even if the daemon just terminated without performing a close on the socket.

Submission

You will submit a single file named **yashd.tgz** that contains all files in a folder by the same name (**yashd**). Make sure there is a **Makefile** that will build your shell with a single command (make). The executables that your makefile should create must be called **yashd** and **yash**. If you have any doubts about submission please post a question on Ed Discussion. It is important that everybody follows these instructions so we can automate the grading process.